# HW7 1.

① $f(x) = \frac{1}{2} x^T Q x + q^T x + c$

1a $\nabla f(x) = Qx + q$ ; $\nabla^2 f(x) = \underline{Q}$.

$$\|\nabla f(x) - \nabla f(y)\| = \|Qx + q - Qy - q\|$$

$$= Q\|x - y\|$$

$\boxed{\text{has } Q\text{-lipschitz gradients}}$

we have $Q = \begin{bmatrix} 13 & 12 & -2 \\ 12 & 17 & 6 \\ -2 & 6 & 12 \end{bmatrix}$, $q = \begin{bmatrix} -22 \\ -14.5 \\ 13 \end{bmatrix}$, $c = -1$

$$\boxed{\|\nabla f(x) - \nabla f(y)\| \leq M \|x - y\|}$$

This defines smoothness

$\Rightarrow$ $Q\|x - y\| \leq M\|x - y\|$

$$\boxed{\frac{Q\|x - y\|}{\|x - y\|}} \leq M \quad \boxed{\|x - y\| \neq 0}$$

$\boxed{\frac{Q\|x - y\|}{\|x - y\|}}$ is the spectral norm for vector $\underline{(x - y)}$.

1(a)

1(a)

$$M = \max_{\|x-y\| \neq 0} \frac{\varphi \|x-y\|}{\|x-y\|} = \max_{z \neq 0} \frac{\|\varphi z\|}{\|z\|}$$

(let $x-y = z$)

when we normalize $z$ to be unit vector

$$\boxed{M = \max_{\|z\|=1} \|\varphi z\|}$$

$\cancel{M^2 = \max \|\varphi^2 z^2}$

$$M^2 = \max \|\varphi z\|^2 = (\varphi z)^T (\varphi z)$$

$$M^2 = \max_{\|z\|=1} z^T \varphi^2 z$$

$\varphi$ is symmetric, The eighet vector of $\varphi$ are orthogonal,

let $\lambda_i$ Eighen values
$v_i$ Eighen vector

$$M^2 = \max_{\|z\|=1} z^T \varphi^2 z = \Sigma$$

~~unit vector $z = \sum a_i$~~

unit vector $z = \sum_i a_i v_i$ ; $\|z\|_2 = 1$

1(a)

$$m^2 = z^T \varphi^2 z = \left(\sum_j a_j v_j\right)^T \varphi^2 \left(\sum_i a_i v_i\right)$$

1(a) $$= \left(\sum_j a_j v_j\right)^T \sum a_i \varphi(\varphi v_i)$$

$$\varphi v_i = \lambda v_i \Rightarrow \left(\sum_j a_j v_j\right)^T \sum a_i \varphi(\lambda_i v_i)$$

$$\Rightarrow \left(\sum_j a_j v_j\right)^T \left(\sum a_i \lambda_i (\varphi v_i)\right)$$

$$\Rightarrow \left(\sum_j a_j v_j\right)^T \left(\sum a_i \lambda_i^2 v_i\right)$$

$$\Rightarrow \left(a_j a_i \lambda_i^2 \underbrace{v_j^T v_i}_{=1}\right)$$

$$\Rightarrow \sum a_i^2 \lambda_i^2$$

$$\boxed{\therefore z^T \varphi^2 z = \sum a_i^2 \lambda_i^2}$$

$$\therefore m^2 = \max_{\|z\|=1} \sum a_i^2 \lambda_i^2$$

$a_i$ are normalized coeff of unit vector $\therefore \sum a_i^2 = 1$

$$m^2 = \max_{\|z\|=1,i} \lambda_i^2$$

$$\Rightarrow \boxed{m = \max_{\|z\|=1,i} \lambda_i}$$ max Eigen values of $\varphi$.

1(a)

So we got

1(a)

$$\boxed{M = \max_{\|Z\|=1} \lambda(\varphi)}$$

Maximum Eighen value of $\varphi$,

$$f(x) = \frac{1}{2} x^T \varphi x + q^T x + c$$

$$\nabla f(x) = \varphi x + q$$

$$\frac{\varphi \|x-y\|}{\|x-y\|} \leq \underline{\underline{M}}$$

$\rightarrow$ $\boxed{\text{Spectral norm}}$

The smoothness parameter is $\underline{M}$

and $M = \max \lambda_i(\varphi)$

when $\lambda_i$ is max Eighen value of $\varphi$.

$\Rightarrow$ np.linalg.eig($\varphi$)

Eighen Values = [27.89, 13.84, 0.25]

1(a) & (b)

$$\lambda_{i(max)} = \underline{27.89}$$

1(a) $\therefore \boxed{M = 27.89}$.

Smoothness Parameter $M = 27.89$

---

$$f(x) = \frac{1}{2} x^T Q x + q^T x + c$$

1(b) $\nabla f(x) = Qx + q^T$

$\boxed{\nabla^2 f(x) = Q.}$ and $Q$ has all Positive Eigen value

$Q$ has Eigen value $= [27.89, 13.84, 0.25]$

$\boxed{Q \text{ is Positive Definite}}$

$\boxed{\therefore f(x) \text{ is Strongly convex}}$

1(b) $\nabla^2 f(x) \geq m I$.

~~minimum Eigen value of~~

$\nabla^2 f(x)$ has smallest Eigen value which bigger than $\underline{\underline{m}}$

$\nabla^2 f(x) = \varphi$.

The Eigen value of $\varphi = [27.89, 13.84, 0.25]$

$$\boxed{\lambda_{i_{min}} = 0.2589}$$

~~$m \geq 0.2589$~~

$$\boxed{\text{strong convexity Parameter } m = 0.2589}$$

1a $f(x)$ is smooth.

smoothness parameter $M \simeq 27.89$

1b $f(x)$ is strongly convex

strong convexity parameter $m \simeq 0.2589$.

```
import numpy as np
A = np.array([[13, 12, -2],
              [12, 17, 6],
              [-2, 6, 12]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"eighen values {eigenvalues}")
print(f"eighen values {np.max(eigenvalues)}")

print(f"eighen vectors {eigenvectors}")
```

```
eighen values [27.89814954  0.25893881 13.84291164]
eighen values 27.898149543827557
eighen vectors [[-0.59441633 -0.65394243 -0.46800484]
 [-0.77421041  0.62272298  0.1132004 ]
 [-0.21741083 -0.42962239  0.87644574]]
```

1

```
ls
```

```
drive/  sample_data/
```

1a) $f(x)$ is smooth.

$M = 27.89$ ( Smoothness Parameter)

1(b) $f(x)$ is strongly convex

$\nabla^2 f(x) > 0$

$m = 0.2589$

## 1. Set up the GPU

First, make sure your colab has access to a GPU.
Select Runtime -> Change runtime type > GPU.

Check to see if this works by running the following cell.

```
import torch

print(torch.cuda.is_available())
from google.colab import drive
drive.mount('/content/drive') #mounts google drive and it becomes
```

Show hidden output

## 2. Sync your code to Colab

```
from IPython.utils.text import re
import math
from math import sqrt
import numpy as np
import matplotlib.pyplot as plt
import random
import warnings
warnings.filterwarnings("ignore")

n = 3  #dim: 5D R5

Q = np.array([[13, 12, -2],
              [12, 17, 6],
              [-2, 6, 12]])
q = np.array([[-22.0, -14.5, 13.0]])
#print(f" Q is {Q} and q is {q}: q_dim {q.shape}")

ALPHA = 0.12 #0.12 #0.15 # 0.1(works)0.2 #  0.2 #0.45
BETA = 0.95 #0.9 (doesnt work well) # 0.8 # 0.92


def get_fn_val_at_x(x_current):
  return (x_current.T@Q@x_current + q@x_current + -1.0).item()

x_star = -np.linalg.inv(Q)@q.T
```

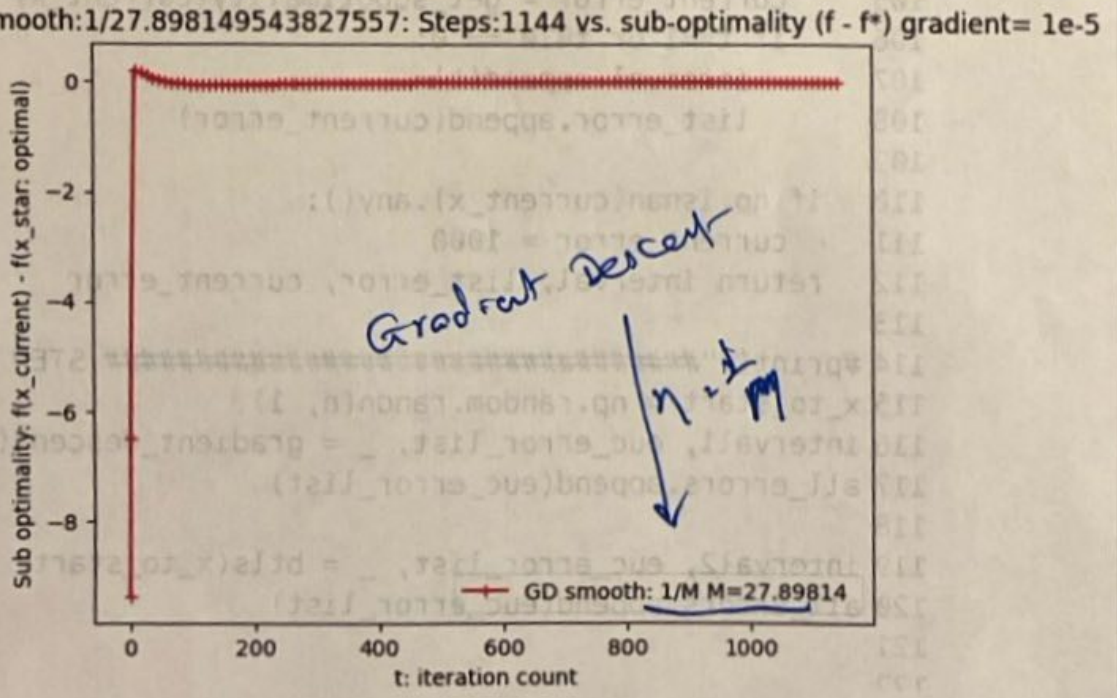
```
f_optimal = get_fn_val_at_x(x_star)
#print(f"f_optimal {f_optimal}")

def get_subotimality(x_current):
  f_current = get_fn_val_at_x(x_current)
  return (f_current- f_optimal)

def current_gradient (x, Q, q):
  cg = Q @ x + q.T
  return cg

eigenvalues, eigenvectors = np.linalg.eig(Q)
M_step = np.max(eigenvalues)
def const_step(multiplier=1.0):
  eigenvalues, eigenvectors = np.linalg.eig(Q)
  #return float(0.1 *multiplier)
  return (1.0/np.max(eigenvalues))

def btls_alpha_beta(current_x, alpha, beta, cg):
  step_size = 0.99999 # 1.0; 0.999999 #random.uniform(0.0, 0.9999
  cg_square = np.linalg.norm(cg)**2
  #next_x = (current_x - step_size * cg)
  current_fx_val = get_fn_val_at_x(current_x)
  #while get_fn_val_at_x(next_x) > (current_fx_val - (alpha*step_
  #q dimension need change leave as is for now use linalg.norm
  for t in range(100):
    next_x = (current_x - step_size * cg)
    if get_fn_val_at_x(next_x) > (current_fx_val - (alpha*step_si
      step_size = step_size*beta
      if step_size < 1e-10: #step size getting too small - get out
        break
  return step_size

all_errors = []
GD_STEPS = 0
def gradient_descent(current_x, Q, q, step_fn, num_steps=10000, mu
  interval = []
  list_error = []
  interval.append(0)
  list_error.append(get_subotimality(current_x))
  for t in range(1, num_steps+1):
    cg = current_gradient(current_x, Q, q)
    if np.linalg.norm(cg) < error_max:
      print(f"GD converged in num_steps = {t}")
      global GD_STEPS
      GD_STEPS = t
      break
    if multiplier is not None:
```

```
      current_x = current_x - step_fn(multiplier) * cg
    else:
      current_x = current_x - step_fn(t) * cg
    current_error = get_subotimality(current_x)
    if t==1 or t%10 == 0:
      interval.append(t)
      list_error.append(current_error)

  if np.isnan(current_x).any():
    current_error = 1000 #set some BIG NUMBER
  #print(f"GD converged to solution: {current_x} with error {curre
  return interval, list_error, current_error

BTLS_STEPS = 0
def btls (current_x, Q, q, step_fn, num_steps=10000, error_max = 1
  interval = []
  list_error = []
  #the is back tracking ALPHA BETA SEARCH
  alpha = float(ALPHA)
  beta = float(BETA)
  interval.append(0)
  list_error.append(get_subotimality(current_x))
  for t in range(1, num_steps+1):
    cg = current_gradient(current_x, Q, q)
    if np.linalg.norm(cg) < error_max:
      print(f"BTLS converged in num_steps = {t}")
      global BTLS_STEPS
      BTLS_STEPS = t
      break
    step_size = step_fn(current_x, alpha, beta, cg)
    current_x = current_x - step_size * cg
    current_error = get_subotimality(current_x)
    if t==1 or t%10 == 0:
      interval.append(t)
      list_error.append(current_error)

  if np.isnan(current_x).any():
    current_error = 1000
  return interval, list_error, current_error

#print(f"################################ STEP = 1/t ###########
x_to_start = np.random.randn(n, 1)
interval1, euc_error_list, _ = gradient_descent(x_to_start, Q, q,
all_errors.append(euc_error_list)

interval2, euc_error_list, _ = btls(x_to_start, Q, q, btls_alpha_b
all_errors.append(euc_error_list)
```

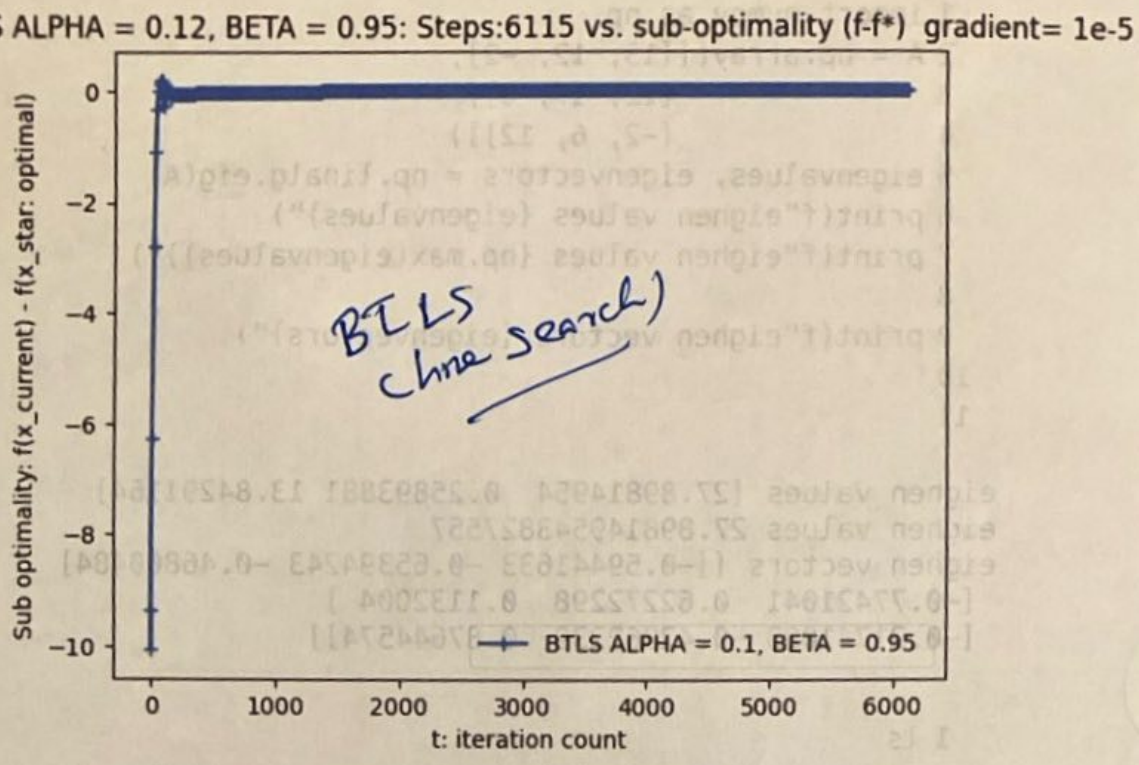
```
#plot the results:
plt.xlabel("t: iteration count")
plt.ylabel("Sub optimality: f(x_current) - f(x_star: optimal)")
plt.plot(interval1, all_errors[0], marker='+', color='red', label=
plt.title(f" GD 1/M smooth:1/{M_step}: Steps:{GD_STEPS} vs. sub-op
plt.legend()
plt.show()

plt.xlabel("t: iteration count")
plt.ylabel("Sub optimality: f(x_current) - f(x_star: optimal)")
plt.plot(interval2, all_errors[1], marker='+', color='blue', label
plt.title(f" BTLS ALPHA = {ALPHA}, BETA = {BETA}: Steps:{BTLS_STEP
plt.legend()
plt.show()
```

P2

```
GD converged in num_steps = 1144
BTLS converged in num_steps = 6115
```



Q2


BTLS ALPHA = 0.12, BETA = 0.95: Steps:6115 vs. sub-optimality (f-f*) gradient= 1e-5
Sub optimality: f(x_current) - f(x_star: optimal)
t: iteration count
BTLS ALPHA = 0.1, BETA = 0.95
0
−2
−4
−6
−8
−10
0
1000
2000
3000
4000
5000
6000
BTLS
(line search)

Q3.

(3) As discussed in lecture

Let us take

$$f(x) = ax^2, \quad a > 0$$

$$\nabla f(x) = 2ax$$

$$\nabla^2 f(x) = 2a$$

$\nabla^2 f(x) > 0$ $\therefore$ $\boxed{f(x) \text{ is Positive Definite}}$

$f(x)$ is convex and PD

Let us write next $x$ as $x_+$

$$x_+ = x - t(\nabla f(x))$$

$$x_+ = x - t(2ax)$$

$x_+$: $x_{next}$, $x$: $x_{current}$

$$\Rightarrow \boxed{x_{k+1} = (1 - 2at)^k x_0}.$$

Different functions can have different value of "a"

# Q3

$$x_{k+1} = (1-2at)^k x_0$$

If $(1-2at) > 1$ or $(1-2at) < -1$

This diverges. ~~However,~~

Ca ① However $\boxed{(1-2at) > 1 \text{ is not possible}}$ because $a > 0$ and $t > 0$.

$(1-2at) > 1$ is degenerate case

NOT valid case

Ca ② $\boxed{(1-2at) < -1}$

Case 2

$\Rightarrow$

$\Rightarrow t > \frac{1}{a}$ or $a > \frac{1}{t}$

$(1-2at)^k$ will diverge to $\infty$

Ca 3:

$\boxed{\text{for } (1-2at) = -1}$

k even $(1-2at)^k > 0$

k odd $(1-2at)^k < 0$

This will result in oscillation of $x$

# Q3

$f(x) = ax^2$, $x^* = 0$, $\Rightarrow$

Q3 ~~$0 < t < \frac{1}{a}$~~

$$x_+ \Rightarrow x - t\,2ax = 0$$

$$t = \frac{1}{2a}$$

$\therefore$ For $\boxed{0 < t < \frac{1}{2a}}$ $\nabla^2 f(x) = 2a$ ~~set~~ $\nabla^2 f(x) > 0$

This function converges $\therefore$ strongly for $0 < t < \frac{1}{2a}$.

$\boxed{\text{However, the curvature "}2a\text{", } (\nabla^2 f(x) = 2a) \text{ changes based on the value of } a.}$

Hence value of "t" is dependent on "a".

$\therefore$ The step size "t" is dependent on "a" ~~the~~ which controls curvature $(\nabla^2 f(x) > 0)$

$\boxed{\text{Hence a single value of "t" will not work for all strongly convex functions of this class.}}$

4a

Q4 if $f_i$ are convex functions

4a show $f(x) := \sup_i f_i(x)$ is convex

let $x \in \mathbb{R}$, then

Definition sup :- Supremum is smallest real number that is greater than or equal to every element in the set.

Given this definition

$$\boxed{\sup_i f_i(x) \geq \max_i f_i(x) \quad \forall i}$$

i.e. $$\boxed{\sup_i f_i(x) \geq f_i(x) \ \forall i}$$

$$\Rightarrow \boxed{\cancel{\sup_i f_i(x) = \max_i f_i(x)}}$$

we can see that ~~sup $f_i(x)$~~

$$\sup f_i(x) = \sup \max f_i(x)$$

# 4a

4a

if $0 \le \theta \le 1$, $x, y \in \text{dom } f$.

$$f(\theta x + (1-\theta) y) \le \theta f(x) + (1-\theta) f(y)$$

$\theta \in [0, 1]$

$\boxed{\text{Reference Boyd: 3.2.3}}$

$$\boxed{f(x) = \sup_i f_i(x)} \quad (1)$$

$$f(\theta x + (1-\theta) y) = \sup_i f_i(\theta x + (1-\theta) y)$$

$f_i$ are convex

$$\therefore \sup_i f_i(\theta x + (1-\theta) y) \le \sup_i (\theta f_i(x) + (1-\theta) f_i(y))$$

$$\underbrace{\sup_i f_i(\theta x + (1-\theta) y)} \le \theta \sup_i f_i(x) + (1-\theta) \sup_i f(y)$$

$$\downarrow$$

$$f(\theta x + (1-\theta) y) \le \theta f(x) + (1-\theta) f(y)$$

(note $f(x) = \sup_i f_i(x)$) (1)

This show (1) $\rightarrow f(x) = \sup_i f_i(x)$ is

– is convex

4b

4(b) (Reference BOYD Section 3.2.3)

4b) The largest eigen value of a symmetric matrix is a convex function.

i.e. ~~f(x)~~ $f(M) = \lambda_{max}(M)$

$$f(M) = \sup\{ y^T M y \mid \|y\|_2 = 1 \}$$

~~$y \in R^m$~~ $y \in R^m$

$M \in S^m$, i.e. $M \in R^{m \times m}$, $M = M^T$

$y \in R^m$

$y^T M y$ = is weighted sum of M (y is unit vector)

* As M changes $f(M)$ follows change in M for fixed y.

Hence ~~linear~~ $f(M) \to y^T M y$ is linear in M

4b

It follows that

$f(M) = \sup\{ y^T M y \mid \|y\|_2 = 1 \}$

4b is also convex because sup of (linear) convex function $y^T M y$ is also a convex function (in ~~4b~~ we show that sup is ~~convex~~ convex)

$$\boxed{\therefore f(M) = \lambda_{max}(M) \text{ is convex}}$$

$f(M) = \lambda_{max}(M) = \sup\{ y^T M y \mid \|y\|_2 = 1 \}$

$\lambda_{max}(\theta M_1 + (1-\theta) M_2) = \sup\{\theta(y^T M_1 y) + (1-\theta)(y^T M_2 y)\}$

$\leq \theta \sup\{\theta(y^T M_1 y)\} + (1-\theta) \sup\{(1-\theta) y^T M_2 y\}$

$$\boxed{f(M) \leq \theta f(M_1) + (1-\theta) f(M_2)}$$

$\therefore \boxed{f(M) = \lambda_{max}(M)}$ is convex

4b

4(b) Has shown that

4b $f(M) = \lambda_{max}(M)$ is convex.

is Largest Eighen value of symmetric matrix is a convex function.

SECOND PART.

Is it true ~~for~~, ie. is Eighen value of largest magnitude convex

ie is $g(M) = \max_i |\lambda_i(M)|$ is covex ?

$\boxed{\text{Yes,}}$ Eighen value of largest magnitude is CONVEX

$g(M) = \max_i |\lambda_i(M)|$ ~~= max(λmax~~

$= \max(\lambda_{max}(M), -\lambda_{min}(M))$

~~Because $|x| \geq 0$, for $x \geq 0$~~

$$g(M) = \max_i |\lambda_i(M)|$$

46

$$= \max(\lambda_{max}(M), -\lambda_{min}(M))$$

Because $|x| = \begin{cases} x & \text{for } x > 0 \\ -x & \text{for } x < 0 \\ 0 & \text{for } x = 0 \end{cases}$

$$\boxed{g(M) = \max_i |\lambda_i(M)| = \max(\lambda_{max}(M), -\lambda_{min}(M))}$$

$\lambda_{max}(M)$ is convex

$-\lambda_{min}(M)$ is convex

$\max(\lambda_{max}(M), -\lambda_{min}(M))$ is convex

$\therefore$ The Eigen value of largest magnitude is convex

4C

4(C) ~~we have seen~~

In a weighted graph,

① Each edge has weight $\underline{w}$
($w$ is a vector)

② The path between two nodes $\underline{a}$ and $\underline{b}$ ⇒ In the path with minimum sum of edge weights among all paths between $\underline{a}$ and $\underline{b}$
↳ and this is $f(\underline{w})$

$$f(w) = \min_i w^T f_i$$

let there be a total of "$i$" paths in total

$$\boxed{f(w) = \min_i w^T f_i}$$

$(f_i)_j \in \{0, 1\}$ ~~for edge~~

~~for edge j if it is part of i~~

$(f_i)_j = 1$ for edge j if $e_j$ is in $Path_i$

$(f_i)_j = 0$ for edge j, if $e_j$ is NOT in $Path_i$

# 4c

The Path cost :- $P(w)$ = Path weight

$$P_i(w) = (w^T f_i) = \sum_{j=1} f(i)_j \, w_j$$

$$(f_i)_j = 1 \quad \text{if } e_j \in P_i$$
$$(f_i)_j = 0 \quad , \text{if } e_j \notin P_i$$

The edge weight are <u>Assumed</u> be non-negative.

$\boxed{\text{BoyD-3.2.1}}$ Reference

<u>Non-negative</u>, scaling and addition of convex function is <u>Convex</u>.

Non-negative, weighted sum of concave function is concave (<u>Dual</u>).

$$f(w) = \min_i (w^T f_i)$$

$P_i$ is linear, it is both concave and convex

Affine function are concave and convex

# 4c HW7

$$f(w) = \min_i (w^T f_i)$$

$w^T f_i$ is linear, Hence concave and convex

for $\forall\, w_1, w_2$ and $\theta\,[0,1]$

$$f(\theta w_1 + (1-\theta) w_2) = \min_i \left( (\theta w_1^T + (1-\theta) w_2^T) f_i \right)$$

$$= \min_i \left( \theta w_1^T f_i + (1-\theta) w_2^T f_i \right)$$

4c/

$$\geq \theta \min_i (w_1^T f_i) + (1-\theta) \min_i (w_2^T f_i)$$

$$\geq \theta f(w_1) + (1-\theta) f(w_2)$$

~~Then~~ As $w^T f_i$ is linear

$$f(\theta w_1 + (1-\theta) w_2) = \theta f(w_1) + (1-\theta) f(w_2)$$

$\therefore$ $f(w) = \min_i (w^T f_i)$ is concave